V2.9 Mood 核心体验优化

> ❕ Mood 是产品当前核心模块，也是扩列的重要基础。相对于一些 BUG 解决，保障 Mood 核心体验优良、流畅是一个优先级更高的事情。

背景

Mood 模块目前存在一定的代码架构和用户体验问题。

代码重构的成本和风险都比较高，希望在 V2.9 上优先解决以上核心体验优化：**视频首帧、卡顿**问题：

1. 视频播放每次都加载转圈一定时间
2. 过场动画缺失，跳转到详情页播放偏生硬
3. 发布慢（图片、视频）
4. 一些核心场景流畅度增强

环境

手机：Redmi Note 4

分支：feature/quick_chat_2.8

分析

预加载列表和视频

保证尽快显示列表

跳转详情页

主要是从交互体验上改进。

过场动画1d

启动优化1d

视频首帧耗时

Groovy

```
// 有缓存，~500ms
06-24 15:58:00.941 13021 13021 D VideoOpt: setPathSrc: /storage/emulated/0/Android/data/com.zebra.letschat/cache/media/video/v0c936g70000c3a3khjc77u106p0if30/1624521460126_8a4d7d7b7d70f1a9ebb84cd333547bea.mp4
06-24 15:58:00.941 13021 13021 D VideoOpt: pause: false
06-24 15:58:00.985 13021 13092 D VideoOpt: onPrepare
06-24 15:58:01.440 13021 13021 D VideoOpt: onRenderStart

// 无缓存，~5.1s
06-24 16:00:44.748 13021 13021 D VideoOpt: setVideoIdSrc: v0c936g70000c39sn1rc77uf41vtmr2g
06-24 16:00:44.748 13021 13021 D VideoOpt: pause: false
06-24 16:00:45.224 13021 13021 D VideoOpt: onPrepare
06-24 16:00:49.821 13021 13021 D VideoOpt: onRenderStart
```

纯文字耗时

首次或者偶尔有 loading

背景的加载会有个闪屏的过程。

Apache

```
// Redmi Note 4
// 加载背景比较晚，从进入到加载完成需要 800ms
17:30:53.503 20450 20450 D VideoOpt: SingleMoodActivity.onCreate
17:30:53.627 20450 20450 D VideoOpt: SingleMoodActivity.onWindowFocusChanged
17:30:53.685 20450 20450 D VideoOpt: SingleUserFragment.onCreateView
17:30:54.135 20450 20450 D VideoOpt: SingleUserFragment.onResume
17:30:54.296 20450 20450 D VideoOpt: SingleUserFragment.load bg start
17:30:54.303 20450 20450 D VideoOpt: SingleUserFragment.load bg success
```

跳转到发布页耗时

图片、视频的跳转耗时有明显的延迟。

CSS

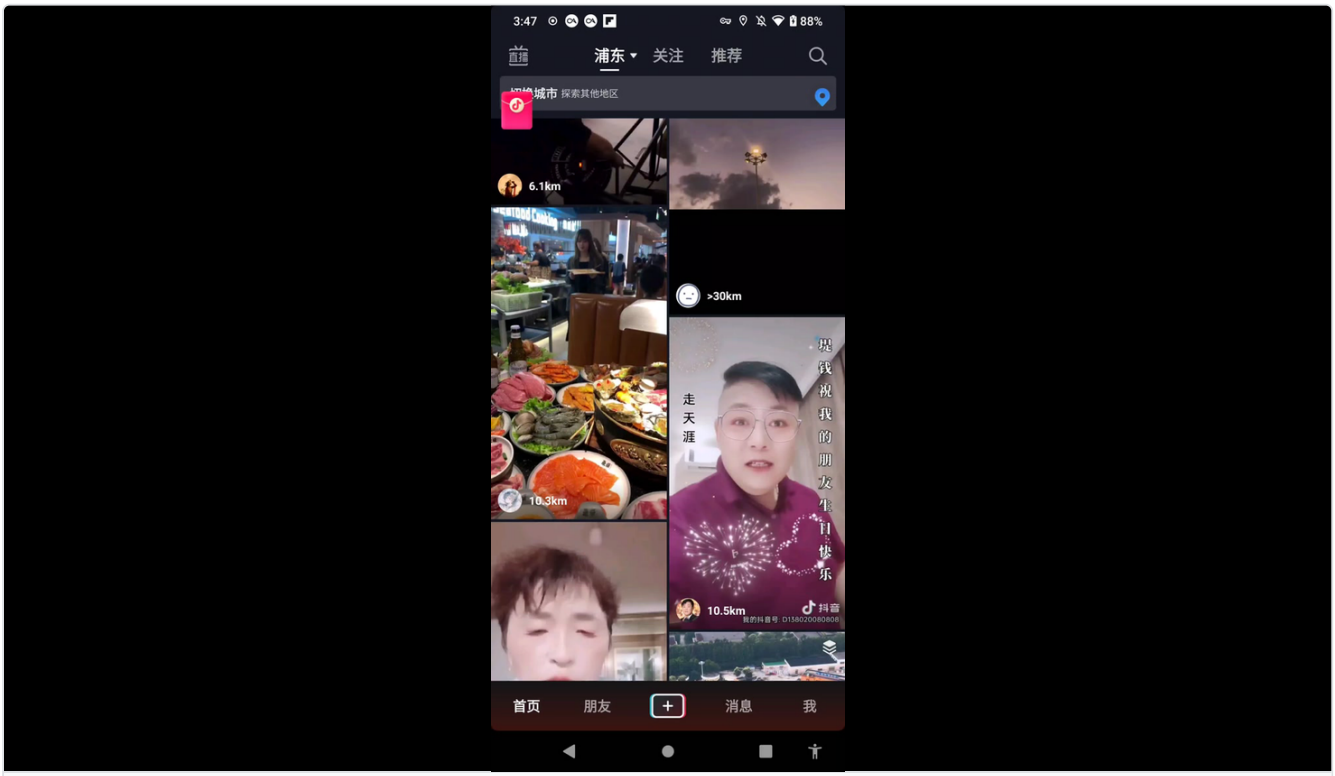
```
// 文字跳转，444ms
06-24 16:30:13.031 21841 21841 D VideoOpt: MoodTypeSelectActivity.mood_select_text_view
06-24 16:30:13.197 21841 21841 D VideoOpt: PublishActivity.onCreate
06-24 16:30:13.475 21841 21841 D VideoOpt: PublishActivity.onWindowFocusChanged

// 图片、视频跳转，1.5s
06-24 16:44:39.080 26592 26592 D VideoOpt: MoodMediaPickerFragment.toPreview
06-24 16:44:39.896 26592 26592 D VideoOpt: MoodMediaPickerFragment.navigationOrToast
06-24 16:44:40.257 26592 26592 D VideoOpt: PublishActivity.onCreate
06-24 16:44:40.269 26592 26592 D VideoOpt: PublishActivity.loadImage.start
06-24 16:44:40.582 26592 26592 D VideoOpt: PublishActivity.onImageLoadSuccess
06-24 16:44:40.583 26592 26592 D VideoOpt: PublishActivity.onWindowFocusChanged
```

竞品分析

抖音

跳到转详情页有个基本的缩放感觉，但没有用共享元素过渡。

返回的时候，走系统默认动画。

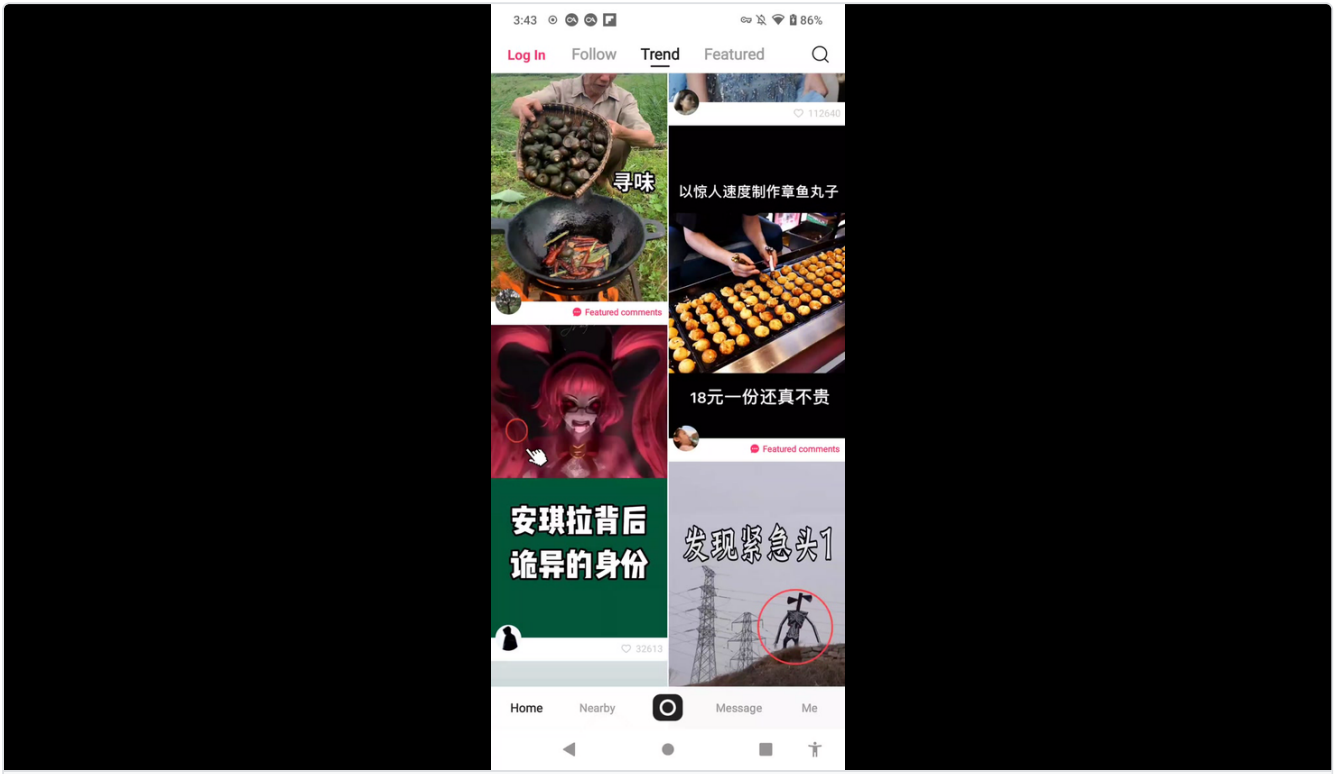


douyin.mp4

快手

跳到转详情页共享元素切换的效果，没有一点闪的感觉。

返回的时候，原路缩回。



kuaishou.mp4

注：虽然快手切换的更自然，不知道为什么总感觉不如抖音切换看着舒服（纯个人感受）。

方案

粗估耗时：6d

注：前期方案阶段简单用日志初步定位问题，具体开发阶段使用 systrace 量化分析问题。

预加载列表和视频

进入首页，默认加载 Mood TAB（当做预加载 explore 列表）。

拿到 explore 列表数据之后，开始预加载列表项中 Mood 视频前部 200KB 内容。

视频缓存具体预加载可参考后面 预加载视频缓存 部分。

跳转详情页

基本实现页面缩放动画（类抖音），让看上去顺畅连贯。

刚好系统的 makeScaleUpAnimation 就是我们的诉求（Arouter 也是支持 ActivityOptions的，不需要做额外适配工作 ）。

Kotlin

```kotlin
val activityOptionsCompat = ActivityOptionsCompat.makeScaleUpAnimation(
    it,
    0 // StartX
    0, // StartY
    it.width,                       // Start Width
    it.height                        // Start Height
)
ARouter.getInstance()
    .build(ACTIVITY_PATH_NAV_BROWSE_SINGLE)
    .withOptionsCompat(activityOptionsCompat)
    .navigation()
```

ActivityOptionsCompat 支持多种动画：

CSS

```
ActivityOptionsCompat.makeCustomAnimation
ActivityOptionsCompat.makeScaleUpAnimation
ActivityOptionsCompat.makeClipRevealAnimation
ActivityOptionsCompat.makeThumbnailScaleUpAnimation
ActivityOptionsCompat.makeSceneTransitionAnimation
```

目前测试下来 makeScaleUpAnimation（参数一定要调对，参考上述代码示例） 比较符合抖音的预期。效果参考线下试验。

视频首帧耗时

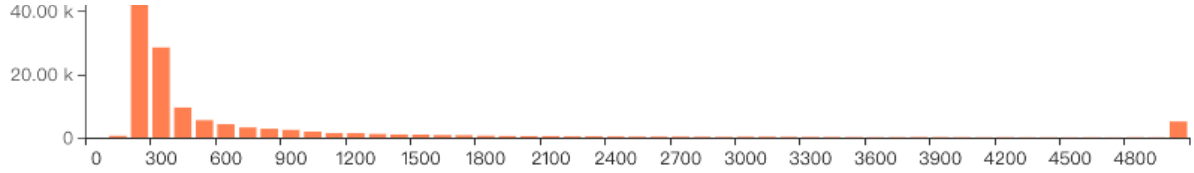
预加载 PlayToken

在点击进入的时候就请求 PlayToken，后面播放的时候直接使用缓存即可。

Kotlin

```kotlin
// 跳转的时候开一个子线程去执行并缓存
ARouter.getInstance()
    .build(ACTIVITY_PATH_NAV_BROWSE_SINGLE)
    .withOptionsCompat(activityOptionsCompat)
    .navigation()
preloadVideoPlayToken()
```

通过查看 multimedia/v1/play_token/{vid} 接口， 大部分耗时在 200-300 ms，如下图：



预期降低 200ms。

启动优化

> 1. setFragment 执行
> 2. getMood 执行
> 3. Music 加载后置

进入之后提前初始化加载的，可提前加载，界面显示的时候尽可能用到前面的数据。

1. setFragment 执行

SingleUserFragment 初始化放在了 *lifecycleScope.launch*(Dispatchers.Main) {} ，会有线程切换的开销。

使用 Dispatchers.Main.immediate 可避免一次线程切换。

CSS

```
// 50ms Dispatchers.Main
07-04 11:20:18.466 19289 19289 D VideoOpt: SingleMoodActivity.onCreate
07-04 11:20:18.486 19289 19289 D VideoOpt: SingleMoodActivity.launchMain.start
07-04 11:20:18.549 19289 19289 D VideoOpt: SingleMoodActivity.launchMain.enter
07-04 11:20:18.574 19289 19289 D VideoOpt: SingleMoodActivity.onWindowFocusChanged

// 2ms Dispatchers.Main.immediate
07-04 11:31:51.399 21288 21288 D VideoOpt: SingleMoodActivity.onCreate
07-04 11:31:51.423 21288 21288 D VideoOpt: SingleMoodActivity.launchMain.start
07-04 11:31:51.425 21288 21288 D VideoOpt: SingleMoodActivity.launchMain.enter
07-04 11:31:51.503 21288 21288 D VideoOpt: SingleMoodActivity.onWindowFocusChanged
```

预期降低 0~10 ms, 主要是减少黑屏

2. getMood 执行

getMood 方法也可以类似优化。

Kotlin

```kotlin
private suspend fun getMood2(): IMMoodConversation? {
    return withContext(Dispatchers.IO) {
        if (localMood == null) {
            try {
                val response: MGetMoodResponseBody = withContext(Dispatchers.Main) {
                    imHandlerWait<MGetMoodResponseBody> {
                        MoodModel.getSingleMood(listOf(moodServerId), it)
                    }
                }
                localMood = response.mood_map[moodServerId]?.let { IMMood.fromServerMood(it) }
            } catch (e: ImErrorException) {
                e.printStackTrace()
            }
        }

        if (localMood != null) {
            return@withContext IMMoodConversation().apply {
                conversationId = localMood!!.conId
                moodList = listOf(localMood!!)
                moodList?.forEach {
                    it.conversationId = localMood!!.conId
                }
            }
        } else {
            return@withContext null
        }
    }
}
```

如果已经 localMood 存在，不需要走线程切换。

CSS

```
// 切换 70ms
07-04 11:43:04.178 24602 24602 D VideoOpt: SingleMoodActivity.onCreate
07-04 11:43:04.212 24602 24602 D VideoOpt: SingleMoodActivity.launchMain.start
07-04 11:43:04.215 24602 24602 D VideoOpt: SingleMoodActivity.launchMain.enter
07-04 11:43:04.215 24602 24602 D VideoOpt: SingleMoodActivity.geMood.start
07-04 11:43:04.287 24602 24602 D VideoOpt: SingleMoodActivity.geMood.end
// 不切换 0ms
07-04 11:46:36.141 25576 25576 D VideoOpt: SingleMoodActivity.onCreate
07-04 11:46:36.178 25576 25576 D VideoOpt: SingleMoodActivity.launchMain.start
07-04 11:46:36.180 25576 25576 D VideoOpt: SingleMoodActivity.launchMain.enter
07-04 11:46:36.181 25576 25576 D VideoOpt: SingleMoodActivity.geMood.start
07-04 11:46:36.181 25576 25576 D VideoOpt: SingleMoodActivity.geMood.end
```

预期降低 0~10 ms, 主要是减少黑屏

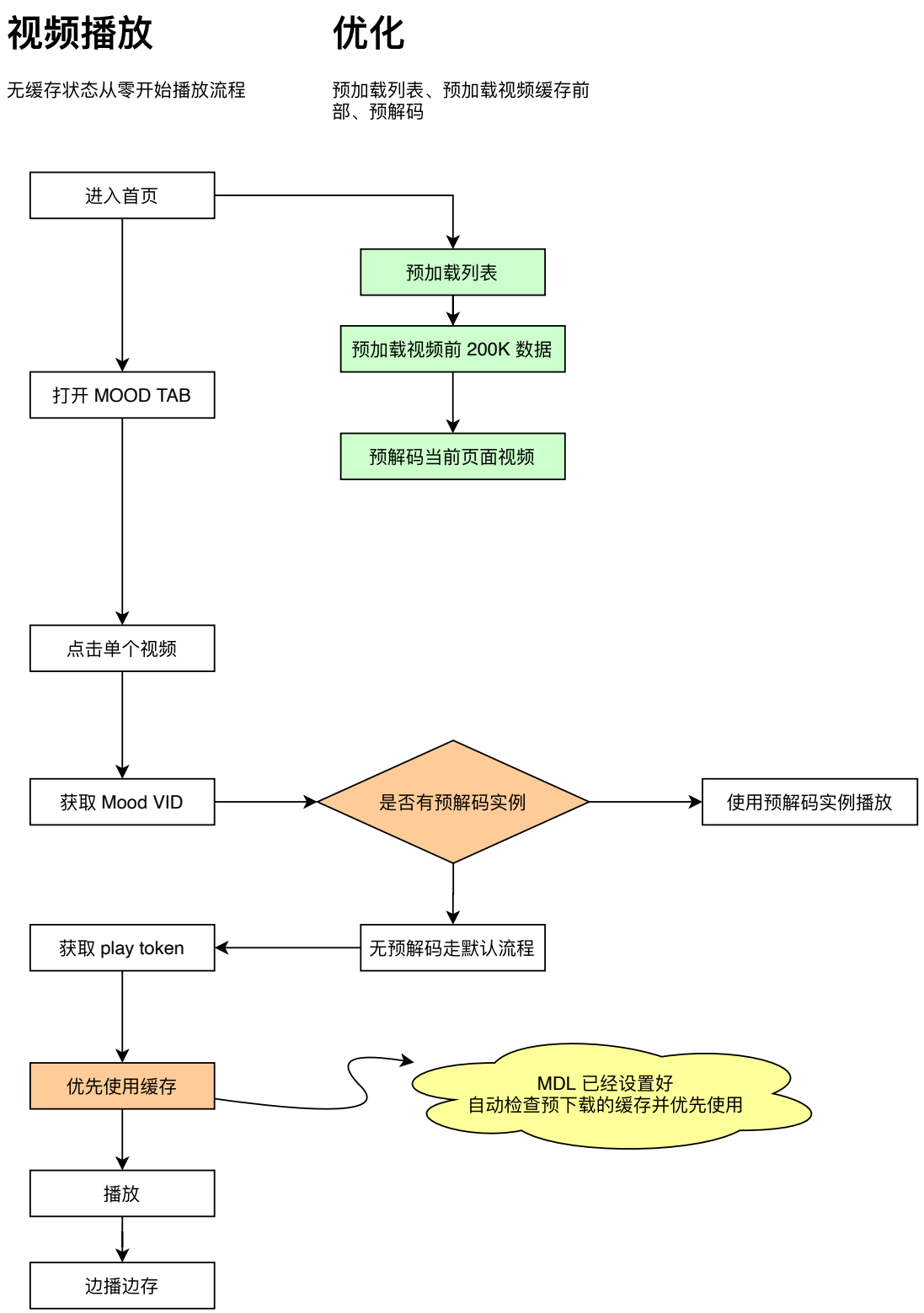
3. Music 加载后置

目前 Mood 加载都强依赖 music 下载好，调整为*并行，分析下来对业务的影响应该是可接受的。*

预期降低 200ms（部分场景）

预加载视频缓存

> ❗ **数据参考**
>
> WIFI 用户比例：15%
>
> Mood 浏览点击比：2%



同时技术上目前实现方式验证下来还是有点问题。

CSS

```
// 无缓存 1707ms
07-04 21:08:29.329 18176 20622 D 播放器 MediaPlayer: ------- mediaPlay.start
07-04 21:08:29.329 18176 20622 D 播放器 MediaPlayer: ------- mediaPlay.end
07-04 21:08:29.731 18176 18176 I 播放器 MediaPlayer: onPrepare
07-04 21:08:29.733 18176 18176 D fengjian: --- usingKey: 7c90217ed7044e742bbe784aa7eef720, hitCacheSize: 0
07-04 21:08:30.811 18176 18176 I 播放器 MediaPlayer: onPrepared
07-04 21:08:31.029 18176 18176 I 播放器 MediaPlayer: onPlaybackStateChanged
07-04 21:08:31.034 18176 18176 I 播放器 MediaPlayer: onLoadStateChanged 1
07-04 21:08:31.036 18176 18176 I 播放器 MediaPlayer: onRenderStart

// 有缓存 691ms
07-04 21:09:36.577 18176 18176 D 播放器 MediaPlayer: ------- mediaPlay.start
07-04 21:09:36.578 18176 18176 D 播放器 MediaPlayer: ------- mediaPlay.end
07-04 21:09:36.623 18176 20711 I 播放器 MediaPlayer: onPrepare
07-04 21:09:36.655 18176 18176 D fengjian: --- usingKey: 7c90217ed7044e742bbe784aa7eef720, hitCacheSize: 1048720
07-04 21:09:37.062 18176 18176 I 播放器 MediaPlayer: onPrepared
07-04 21:09:37.263 18176 18176 I 播放器 MediaPlayer: onPlaybackStateChanged
07-04 21:09:37.266 18176 18176 I 播放器 MediaPlayer: onLoadStateChanged 1
07-04 21:09:37.268 18176 18176 I 播放器 MediaPlayer: onRenderStart
```

预期收益 300~1000 ms

具体实现 Vid 预加载的核心代码如下：

Kotlin

```kotlin
// 添加任务，开始预加载
val vidItem = PreloaderVidItem(mVid, Resolution.SuperHigh, 200 * 1024, false)
TTVideoEngine.addTask(vidItem);
```

列表 Mood 曝光的时候开始预加载当前页面视频。

Kotlin

```kotlin
// 曝光监听
recyclerViewExposeUtil.setRecyclerItemExposeListener(this, object: OnItemExposeListener {
    override fun onItemViewVisible(visible: Boolean, position: Int) {
        if (visible) {
        }
    }
})
```

预解码视频缓存

点击之前新建实例 MediaPlayer 预解码缓存到内存中，使用的时候从内存中直接获取。

实现一个 MediaPlayPool 类。

Kotlin

```kotlin
// MediaPlayPool.kt
fun add(vid: String): MediaPlayer
fun get(vid: String)
fun clear()
```

预期收益 300 ms

纯文字耗时

理论上文字和背景的显示是可以做到无闪的，因为数据本地都有，也可以做到预加载到内存。

主要是背景需要流畅的显示出来，可以在进入页面加载前先从预加载到内存，显示的时候直接显示。

预期收益 100ms

跳转到发布页耗时

确定问题，按 BUG 处理。

链路分析耗时点，文字可以做到快很多，说明图片视频存在明显的代码阻塞（sleep、post）行为。

去掉对应的主动阻塞行为，并解决当时引入时要解决的问题。

Kotlin

```kotlin
override fun toPreview(position: Int, mediaItem: MediaItem) {
    val activity = activity ?: return
    val mediaPickShareData = ActivityShareDataMgr
            .getShareData(activity, token, MediaPickShareData::class.java)
    mediaPickShareData?.mediaSelectModel = mediaSelectModel
    OtherMonitor.monitorMoodPhotoClick()
    val runnable = Runnable { navigationOrToast(mediaItem) }
    PostUtil.remove(runnable)
    PostUtil.postDelay(runnable, 800)
    activity.onResumeRemoveToken(token)
}
```

当时是为了解决快速点击跳转多个问题，应该在点击的地方处理即可，不应该在这里延迟。

Kotlin

```kotlin
// 800 -> 1200
itemView.setOnClickListener(debounceClick (1200) {
    onItemClickListener?.invoke(it, adapterPosition, mediaItem)
})
```

预期收益 800ms（发布页图片视频）

StictMode 卡顿

列表

其他

埋点梳理。

配置项

Settings

explore_preload_enabled - true（仅限制 explore）

mood_explore_preload_size - explore 预加载前部大小

mood_home_preload_size - mood 预加载前部大小

SharePreference

暂无

预期

预期结果是流畅。可以提供一个粗估的量化指标（Redmi Note4）。

PS: 设备越低端收益越大，越高端收益越小。

场景	现状	预期	
跳转到详情页	Loading 慢	缩短loading 增加转场动画 无闪动切换	
视频首帧	有缓存 500ms 有缓存 2000ms~5100ms	有缓存不变 无缓存 1000ms	
文字背景	800ms	500ms且不闪烁	
跳转到发布页	1.5s	0.5s	

效果

优化项及优化效果如下：

【修复】修复部分视频缓存校验导致失效问题，已上线 2.8

【优化】优化 explore 列表卡片和头像偶尔闪动问题，已上线 2.8

【优化】

参考

- 从 2G 到 5G，分分钟看懂通讯产业
- 客户端卡顿埋点验证Layout
- 预加载